KING MONGKUT'S INSTITUTE OF TECHNOLOGY LATKRABANG

SCHOOL OF ENGINEERING

01276390 Computer Engineering Project Preparation (CEPP)

Project
An AI Chatbot for answering questions regarding budget disbursement and procurement at KMITL.

GROUP: CEIPP-68-12

INSTRUCTED BY: Asst. Prof. Dr. Amnach Khawne

Submitted by

Kanyanat Nopparat (66011020)

Ratthabhumi Peansukmanee (66011198)

Jhantharas Toojinda Suttivanich (66011005)

Date of Submission: April 3, 2026

Chapter 1

Introduction

1.1 Background and Motivation

Public sector institutions in Thailand operate under a strict and multi-layered framework of financial regulations. At King Mongkut's Institute of Technology Ladkrabang (KMITL), administrative and finance personnel are required to follow the Government Procurement and Supply Management Act B.E. 2560, the Ministry of Finance Procurement Regulations, the Fiscal Discipline Act B.E. 2561, and a collection of institutional operational manuals. These documents collectively govern every aspect of how the university plans, approves, executes, and records its budget expenditures.

Despite the availability of these documents, accessing the right information at the right moment remains a persistent challenge. The regulatory corpus is large, spanning over a dozen major PDF files written primarily in Thai, and the content is both technical and deeply procedural. A staff member who needs to verify a procurement threshold, confirm a required attachment for an expense claim, or understand the correct e-Market process for a given purchase value must manually search through multiple documents, often without a clear starting point. This process is not only time-consuming but also prone to human error, since regulations are frequently referenced across different files and procedural steps can be easily overlooked.

In recent years, the emergence of large language models (LLMs) and Retrieval-Augmented Generation (RAG) systems has opened a practical path toward automating this kind of document-based question answering. Unlike a simple keyword search, a RAG system understands the meaning of a question, retrieves the most relevant passages from a document corpus, and generates a coherent, cited answer in natural language. This makes it well-suited to regulatory environments where questions are often nuanced and answers must be traceable to official sources.

Motivated by these developments, this project proposes the development of a Thai-language AI chatbot specifically designed for KMITL's budget disbursement and procurement domain. The system is intended to serve as an always-available, accurate, and source-cited

assistant for institutional staff, reducing their reliance on manual document searches and helping them carry out financial procedures with greater confidence and correctness.

1.2 Problem Statement

The central problem this project addresses can be summarised in three interconnected dimensions, each of which directly affects the operational efficiency and regulatory compliance of KMITL's administrative functions.

First, budget disbursements are frequently delayed. When staff encounter an unfamiliar procedure or an ambiguous regulation, they must pause their work to search manually through the relevant documents or consult a senior colleague. This search process is rarely straightforward, as the answer to a single question may require cross-referencing information from multiple files. The resulting delays accumulate across teams and departments, slowing down procurement cycles and disrupting project timelines.

Second, disbursements are sometimes carried out incorrectly. The complexity of the regulatory framework means that even experienced personnel occasionally misinterpret a rule, use an outdated procedure, or omit a required supporting document. These errors can lead to rejected reimbursement claims, requests for revision, and in more serious cases, compliance violations that carry institutional consequences. The root cause is not a lack of diligence but a lack of convenient access to reliable, up-to-date procedural guidance.

Third, the information itself is difficult to locate. KMITL's regulatory and procedural knowledge is stored exclusively in PDF files, with no unified search interface, no structured indexing, and no mechanism for staff to ask questions in plain language. Newer employees, in particular, have no efficient way to build familiarity with the documents and must rely heavily on institutional knowledge passed down informally, which is neither scalable nor consistently accurate.

Together, these three problems point to a clear need for a system that makes regulatory knowledge immediately accessible, contextually accurate, and verifiably grounded in official documents.

1.3 Objectives

This project is guided by the following specific objectives:

1. To design and implement a Retrieval-Augmented Generation (RAG) pipeline capable of ingesting, indexing, and retrieving information from KMITL's Thai-language regulatory PDF documents.
2. To develop a conversational chatbot interface that allows institutional staff to pose questions in natural Thai and receive accurate, clearly reasoned answers in return.
3. To ensure that every system response includes explicit citations identifying the source document and page number, enabling users to verify answers against the original regulatory text.
4. To evaluate the system's retrieval quality using standard information retrieval metrics, including Recall, Precision, and Mean Reciprocal Rank, and to assess end-to-end answer quality using an LLM-as-a-Judge framework modelled on the RAGAS methodology.
5. To deliver a functional and deployable solution that operates at zero ongoing cost by leveraging locally-hosted embedding models and a free-tier language model API.

1.4 Scope of the Project

This project is focused specifically on the budget disbursement and procurement domain of KMITL. The knowledge base is constructed from a defined set of official regulatory and operational documents. The system is designed for use by KMITL administrative and finance personnel and is not intended as a general-purpose legal or financial advisory tool.

The following boundaries define the scope of this work:

Dimension	Scope
Domain	Budget disbursement and procurement procedures at KMITL
Language	Thai-language input and output; regulatory documents are primarily in Thai
Document Sources	Defined set of official PDFs, covering laws, ministerial regulations, and KMITL operational manuals
Users	KMITL administrative and finance personnel
Deployment	Streamlit web application, intended for internal institutional use
Not in Scope	Real-time data integration, financial transaction processing, legal advice, or documents outside the defined corpus

The system does not connect to any live financial system or database. It provides informational guidance only, and users are expected to refer to official documents and consult responsible officers for final decisions on any consequential financial matter.

1.5 Expected Outcomes

Upon successful completion of this project, the following outcomes are anticipated:

- A fully functional RAG-based chatbot application, deployable via Streamlit, that provides Thai-language answers to questions about KMITL procurement and disbursement procedures.
- A vector database built from KMITL's regulatory PDF corpus, enabling fast and semantically accurate document retrieval.
- Measured retrieval performance with a Mean Reciprocal Rank (MRR) above 0.30 and a cosine similarity score above 0.60, confirming that the system consistently locates relevant content.
- End-to-end answer quality scores of at least 4.0 out of 5.0 across faithfulness, answer relevance, and context precision as assessed by the LLM-as-a-Judge evaluation framework.
- A practical reduction in the time required for staff to locate procedural guidance, with all answers accompanied by verifiable source citations.
- A replicable evaluation methodology, including a golden dataset and automated scoring scripts, that can be extended as the document corpus grows.

1.6 Contributions

This project makes the following contributions to both the institution and the broader field of applied natural language processing:

From a practical standpoint, the system addresses a genuine operational gap at KMITL by making institutional regulatory knowledge queryable in natural language. To the best of the authors' knowledge, no comparable AI-assisted document retrieval system currently exists for KMITL's procurement and disbursement domain. The system is also designed to be maintainable and extensible, so new documents can be added to the corpus and the vector database rebuilt without modifying the core application code.

From a technical standpoint, the project demonstrates the feasibility of building a high-quality, zero-cost RAG system for a low-resource language (Thai) in an institutional regulatory context. The use of a locally-hosted multilingual embedding model, instead of a paid embedding API, ensures that the system remains cost-free regardless of query volume. The integration of an LLM-as-a-Judge evaluation framework adapted from the RAGAS methodology provides a principled and reproducible approach to assessing answer quality in the absence of large human-annotated evaluation sets.

Finally, the project contributes a curated golden dataset of representative procurement and disbursement questions, which can serve as a foundation for future evaluation and benchmarking as the system is extended.

Chapter 2

Theoretical Background

2.3 Retrieval-Augmented Generation (RAG)

2.3.1 RAG Paradigm and System Architecture

The Retrieval-Augmented Generation (RAG) framework is implemented to enhance the Large Language Model (LLM) by grounding its responses in external, domain-specific knowledge. For the KMITL Budget AI Chatbot, the architecture operates in two main phases: data ingestion and question-answering. During ingestion, official KMITL budget and procurement regulations (PDF format) are processed, split into manageable chunks, and converted into mathematical vector representations stored in a local ChromaDB vector database. Upon receiving a user query, the system retrieves the most highly relevant document chunks and passes them as context to a Generative AI model (such as Gemma-3 or Gemini) to formulate a precise, policy-compliant response. This restricts the model from generating hallucinated information and ensures all answers strictly adhere to university regulations.

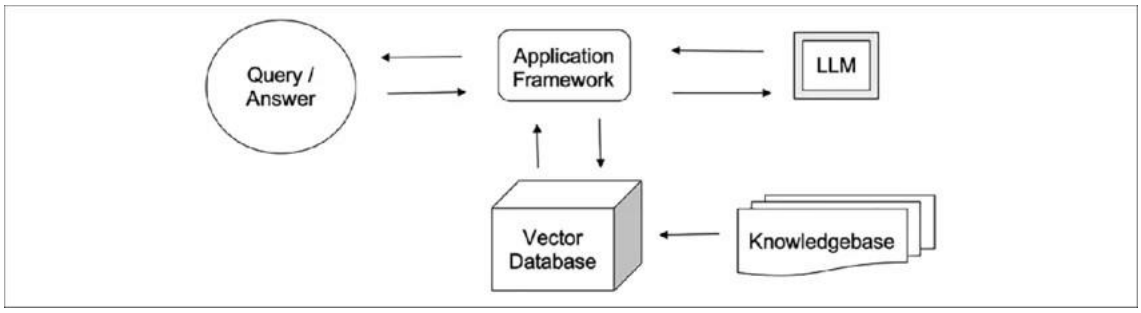

Figure 2.3.1: RAG Architecture Diagram (User -> Retriever -> Generator)

2.3.2 Vector Embeddings and Semantic

Search To enable the system to understand the underlying context and intention of user queries rather than relying on exact keyword matching, vector embeddings are utilized. Text data is transformed into high-dimensional vectors, enabling semantically similar concepts to be clustered together in the vector space. When a user inputs a question, semantic search is executed by calculating the cosine similarity between the query vector and the document vectors within the database. This allows the system to accurately identify and retrieve the correct sections of the budget regulations, even if the user phrases their inquiry differently from the official documentation.

2.3.3 Chunking Strategies and Context Preservation

Processing lengthy and intricate legal procedural documents requires an effective chunking strategy to ensure the retrieved context is both highly relevant and fits within the LLM's context window. The system employs a recursive character text splitting method (RecursiveCharacterTextSplitter) with a chunk size of 500 characters and a 100-character overlap. The overlap is particularly crucial for context preservation; it prevents the loss of critical information or grammatical context that might otherwise be severed at the boundaries of adjacent text chunks. This guarantees that the LLM receives complete and coherent policy clauses to base its answers upon.

2.4 Multilingual Natural Language Processing

2.4.1 Multilingual Sentence Embedding Models

Since the KMITL budget regulations are written entirely in Thai, the system utilizes robust multilingual sentence embedding models, specifically the paraphrase-multilingual-MiniLM-L12-v2 model from HuggingFace. This model has been extensively trained on diverse linguistic datasets, allowing it to efficiently capture the semantic nuances and contextual meanings of Thai text without relying on external API calls. Utilizing a localized embedding model ensures fast retrieval latency and maintains strict data privacy for the university's internal documents.

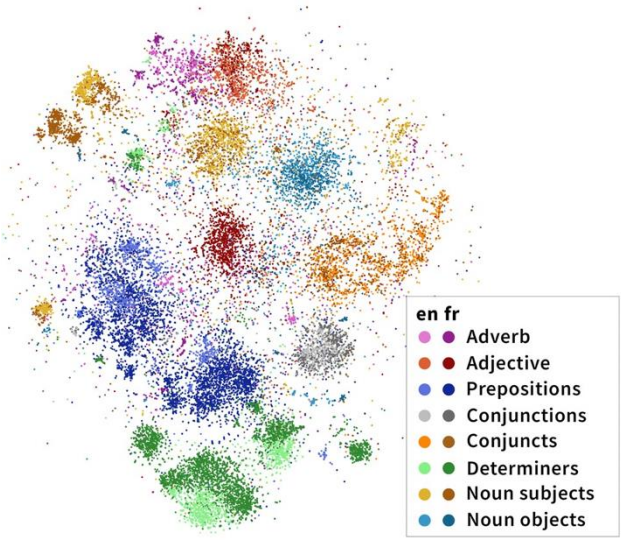

Figure 2.4.1: Multilingual Vector Space Visualization

2.4.2 Challenges of Thai Text Processing

Implementing RAG systems for the Thai language introduces unique Natural Language Processing (NLP) challenges. Unlike English, the Thai language lacks explicit word boundaries or regular spacing between words, making word tokenization and text splitting significantly more complex. Furthermore, official government and university documents often feature dense, formal vocabulary and lengthy sentence structures. By combining strategic recursive text splitting with a powerful multilingual embedding model, the system effectively mitigates these typographical challenges, ensuring that the semantic integrity of the Thai regulations is fully preserved during retrieval.

2.5 Evaluation Frameworks for RAG Systems

2.5.1 Information Retrieval Metrics: Recall, Precision, and MRR

To systematically assess the accuracy of the semantic search component, the system utilizes standard Information Retrieval (IR) metrics evaluated against a custom golden dataset. Recall@K measures the proportion of relevant documents successfully retrieved within the top K results, ensuring no critical rules are missed. Precision@K evaluates the proportion of

retrieved documents that are actually relevant to the query, reducing noise. Finally, Mean Reciprocal Rank (MRR) calculates how high the first relevant document appears in the search results. Together, these metrics ensure the retrieval pipeline reliably fetches the exact required regulations.

2.5.2 LLM-as-a-Judge and the RAGAS Methodology

Evaluating the final generated answers requires analyzing qualitative aspects that traditional metrics cannot capture. Inspired by the RAGAS (Retrieval Augmented Generation Assessment) methodology, the project leverages an automated "LLM-as-a-Judge" grading framework to score the chatbot's performance. The evaluation strictly focuses on three primary dimensions: Faithfulness (ensuring the answer does not hallucinate and is entirely derived from the retrieved documents), Answer Relevance (verifying that the response directly addresses the user's question without filler), and Context Precision (confirming the retrieved text actually contains the correct answer). This automated evaluation pipeline ensures the continuous improvement and reliability of the KMITL Budget AI Chatbot.

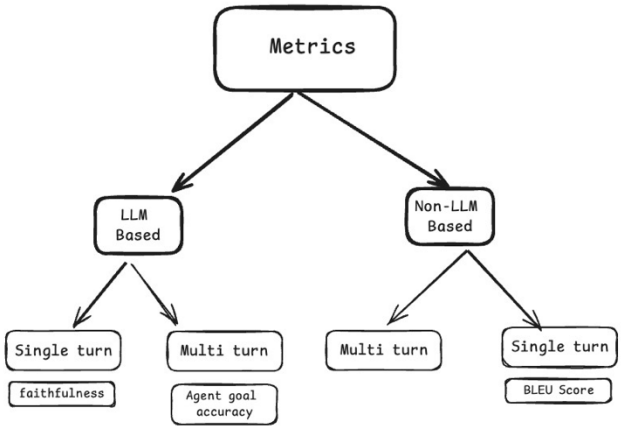

Figure 2.5.2: RAGAS Evaluation Metrics (Faithfulness & Relevance)

Chapter 3

Knowledge Base and Document Corpus

3.2 Document Preprocessing Pipeline

To transform raw institutional regulations into a format suitable for semantic retrieval, a multi-stage preprocessing pipeline was implemented. This ensures that the dense legal language of KMITL's procurement and budget policies is efficiently indexed and accessible by the RAG system.

3.2.1 PDF Text Extraction with PyMuPDF

The source corpus, consisting of KMITL's official regulatory announcements, was primarily stored in PDF format. Text extraction was performed using PyMuPDF (via the PyMuPDFLoader integration in LangChain). Unlike standard PDF parsers, PyMuPDF was selected for its superior performance in handling complex document layouts and its high fidelity in extracting Thai glyphs without introducing significant character corruption, which is critical for maintaining the legal accuracy of retrieved information.

3.2.2 Text Chunking and Overlap Strategy

Due to the context window limitations of Large Language Models and the need for high-precision retrieval, the extracted text was undergo a segmentation process. We employed the RecursiveCharacterTextSplitter with the following parameters:

Chunk Size: 500 characters
Chunk Overlap: 100 characters
This strategy utilizes a "sliding window" approach. The overlap ensures that semantic context—such as the specific regulation number appearing at the beginning of a section or the subject of a sub-clause—is preserved between adjacent chunks. This prevents the loss of vital metadata that would occur if the text were split abruptly at the chunk boundary.

3.3 Corpus Statistics and Coverage Summary

The current knowledge base comprises a curated collection of KMITL's fiscal and procurement regulations. By decomposing these documents into granular segments, the system achieves a high degree of coverage across the following domains:

- Procurement Thresholds: Detailed rules regarding purchasing limits and approval authorities (e.g., Department level vs. Institute level).
- Disbursement Procedures: Step-by-step guidance on domestic and international travel expenses, honorary fees, and reimbursement documentation.
- General Administrative Budgeting: Regulations governing the usage of central funds and specialized faculty-level budget allocations.

The resulting vector database (ChromaDB) currently manages hundreds of distinct semantic nodes, ensuring that even minor regulatory amendments are indexed and available for retrieval.

Chapter 4

Project Timeline and Planning

This chapter presents the chronological development plan of the project, spanning from late November 2025 through early April 2026. The timeline is organized into twenty phases that progress logically from initial orientation and topic selection through system design, development, evaluation, and final presentation. Academic obligations, including the midterm and final examination periods, are incorporated as scheduled interruptions and were accounted for in the overall planning to ensure realistic delivery targets throughout the project lifecycle.

4.1 Development Phases

Phase	Period	Key Milestone
1	27 Nov – 3 Dec 2025	Project documentation reviewed; groups formed
2	4 – 10 Dec 2025	Orientation completed; topic selected
3	11 – 15 Dec 2025	Advisor secured; topic formally submitted
4	16 – 22 Dec 2025	Problem Definition
5	23 – 29 Dec 2025	Literature Review
6	30 Dec 2025 – 5 Jan 2026	Architecture Diagram and system workflow
7	6 – 12 Jan 2026	Detailed design documentation
8	13 – 18 Jan 2026	Technology stack and toolchain confirmed
9	13 – 22 Jan 2026	Midterm examination period
10	23 – 28 Jan 2026	Core development commenced
11	29 Jan – 2 Feb 2026	Evaluation system implemented
12	3 – 9 Feb 2026	System tested; preliminary advisor review
13	10 – 15 Feb 2026	Report revised per advisor feedback
14	16 – 22 Feb 2026	System verification and refinement
15	23 Feb – 1 Mar 2026	Second advisor consultation; further refinement
16	2 – 8 Mar 2026	Final system polish; formal report and slides begun
17	9 – 15 Mar 2026	Draft report and slides reviewed by advisor
18	16 – 23 Mar 2026	Final examination period
19	24 – 31 Mar 2026	Report and slides finalized; Q&A preparation
20	1 – 2 Apr 2026	Final presentation delivered

Phase 1: Project Documentation and Group Formation

27 November – 3 December 2025

The project began with a careful reading of the topic documentation distributed by the faculty. During this period, the team familiarized itself with the overall project scope and requirements, and group members were formally organized to establish the working structure for the remainder of the project.

Phase 2: Orientation and Topic Selection

4 – 10 December 2025

A formal orientation session was conducted, during which candidate project topics were evaluated and a topic was selected. Preliminary technical research was undertaken in parallel to assess the feasibility of the chosen direction and to identify the foundational concepts that would shape the subsequent design work.

Phase 3: Advisor Appointment and Topic Submission

11 – 15 December 2025

A prospective academic advisor was identified and approached during this phase. The team studied the selected topic in greater depth before making a final decision, and the confirmed topic was formally submitted to the faculty by 15 December 2025.

Phase 4: Problem Definition and Chapter 1 Drafting

16 – 22 December 2025

With the topic confirmed, the team turned its attention to articulating the problem the project would address. Chapter 1 was drafted to cover the project background, the significance of the identified problem, a clearly formulated Problem Statement, and the defined objectives, scope boundaries, and expected outcomes of the work.

Phase 5: Literature Review

23 – 29 December 2025

Between five and ten relevant research papers and comparable projects were surveyed during this phase. The team synthesized the key concepts, methodologies, and limitations of prior work and documented these findings to establish the academic grounding necessary before design work could begin.

Phase 6: High-Level System Architecture Design

30 December 2025 – 5 January 2026

The overall system architecture was designed in consultation with the academic advisor. The principal components of the system were identified and their responsibilities defined. An Architecture Diagram was produced to illustrate the end-to-end data flow and processing pipeline, and the system workflow was formally established to guide subsequent implementation work.

Phase 7: Detailed System Design

6 – 12 January 2026

Building on the high-level architecture, this phase elaborated the component-level design in greater detail. Data structures, interface contracts, and individual module responsibilities were specified. The architecture was refined in response to practical feasibility considerations identified during this analysis, resulting in a design documentation set sufficient to guide development.

Phase 8: Technology and Toolchain Selection

13 – 18 January 2026

The programming language, development frameworks, and chatbot-specific technologies were evaluated and selected during this phase. The development toolchain was finalized, the project environment was planned, and an additional consultation with the academic advisor was held to validate these choices before development commenced.

Phase 9: Midterm Examination Period

13 – 22 January 2026

This period coincided with the faculty midterm examination schedule. Project activities were accordingly reduced during this time to allow team members to meet their academic obligations, with a return to full project engagement planned immediately following the examination period.

Phase 10: Core Development Commencement

23 – 28 January 2026

Following the midterm period, the team began executing the development plan that had been established in the preceding phases. Core system components were built in accordance with the detailed design documentation, with each module developed and tested incrementally.

Phase 11: Evaluation System Implementation

29 January – 2 February 2026

The evaluation framework was implemented during this phase, producing the automated testing and scoring infrastructure required to assess system output quality. This work resulted in the evaluate.py and ai_evaluator.py modules along with the curated golden dataset used for LLM-as-a-Judge scoring.

Phase 12: System Testing and Preliminary Advisor Review

3 – 9 February 2026

The system was tested against the defined evaluation criteria and a preliminary progress report was presented to the academic advisor in an informal review session. Feedback and recommendations received during this meeting were recorded to inform the subsequent refinement work.

Phase 13: Report Revision

10 – 15 February 2026

The project report was revised in response to the advisor feedback gathered during the preceding phase. Technical content was expanded where required, and the formatting, terminology, and writing style were standardized throughout the document to improve coherence and overall quality.

Phase 14: System Verification and Refinement

16 – 22 February 2026

A thorough verification pass was conducted across the system to confirm that all components functioned as specified. Issues identified during verification were resolved and further refinements were applied to bring the system to a stable, production-ready state.

Phase 15: Second Advisor Consultation and Further Refinement

23 February – 1 March 2026

The refined system was brought back to the academic advisor for a second consultation. Additional recommendations were incorporated, and the system was further improved in response to this guidance to ensure it met the expected standard of completeness and correctness.

Phase 16: Final System Optimization and Formal Report Production

2 – 8 March 2026

Final optimizations were applied to the system to maximize performance and usability. In parallel, work began on the formal project report and the presentation slide deck, with content drawn from the accumulated documentation and evaluation results produced across previous phases.

Phase 17: Draft Review with Advisor

9 – 15 March 2026

The draft report and presentation slides were submitted to the academic advisor for review. Feedback received during this consultation was used to guide the final round of revisions before the documents were prepared for formal submission.

Phase 18: Final Examination Period

16 – 23 March 2026

Project activities were again temporarily reduced to accommodate the faculty final examination schedule during this period.

Phase 19: Report and Slide Finalization and Presentation Preparation

24 – 31 March 2026

The project report and presentation slides were finalized and prepared for submission. Anticipated questions from the examination panel were reviewed and rehearsed, and the academic advisor was consulted on presentation strategy and areas of expected scrutiny to ensure the team was well prepared for the formal assessment.

Phase 20: Final Presentation

1 – 2 April 2026

All recommendations and refinements gathered throughout the project were applied in the final preparation for the formal presentation. The team delivered the project to the examination panel, drawing on the preparation undertaken in the preceding phase to present the work confidently and completely.

Chapter 5

System Design and Architecture

5.1 System Overview

The KMITL Budget AI Chatbot is designed as a Retrieval-Augmented Generation (RAG) system. Its primary purpose is to bridge the gap between static, complex regulatory documents and end-user inquiries. Instead of relying solely on the pre-trained knowledge of a Large Language Model (LLM)—which may lack specific, up-to-date institutional rules—this system dynamically retrieves relevant document excerpts and uses them to ground its responses, ensuring high accuracy and legal compliance.

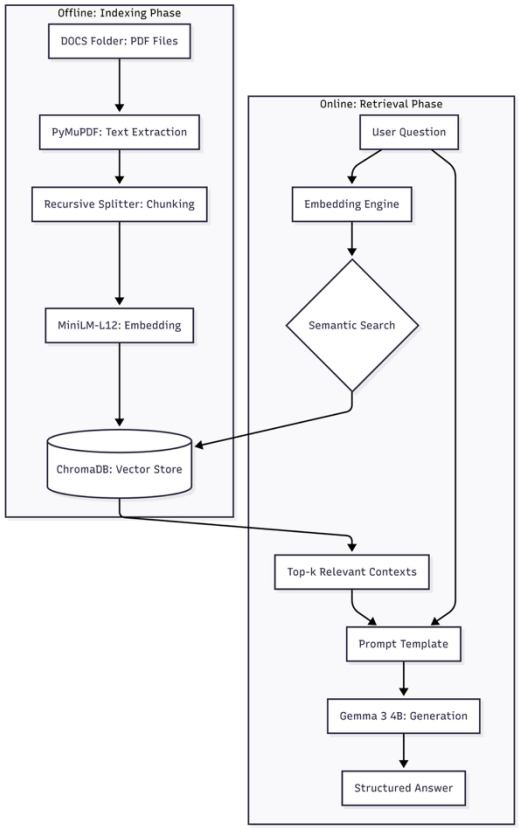

Figure X: System Architecture of the KMITL Budget AI Chatbot (RAG Workflow)

5.2 RAG Pipeline Design

The core of the system is the RAG Pipeline, which consists of two main phases: the Offline Indexing Phase and the Online Retrieval Phase.

5.2.1 Document Ingestion and Indexing

The system automatically monitors the ./Docs directory for new PDF regulations. During initialization, documents are parsed using the PyMuPDF engine. To ensure that the retrieved context is manageable, the text is split into overlapping chunks, ensuring that no regulatory detail is lost at the boundaries of the text segments.

5.2.2 Embedding Generation

Every text chunk is converted into a high-dimensional vector (384 dimensions) using the paraphrase-multilingual-MiniLM-L12-v2 model. This specific model was chosen for its bilingual proficiency, allowing the system to understand semantics in both Thai and English, which is vital for Thailand's institutional context.

5.2.3 Vector Database Construction with ChromaDB

The generated vectors, along with their original text and metadata (source file and page number), are persisted in an on-disk ChromaDB instance. This allows the system to reuse the indexed data without reprocessing the entire corpus every time the application restarts, significantly improving system startup latency.

5.2.4 Query Processing and Retrieval

When a user asks a question, it is transformed into a vector using the same embedding model. The system then performs a Semantic Similarity Search against ChromaDB, retrieving the top 6 (k=6) most relevant context chunks. This "Semantic" approach ensures that the system understands the intent behind the question, even if the user uses different terminology than the official documents.

5.2.5 Answer Generation with Gemma 3 4B

The retrieved context is injected into a specialized System Prompt. This prompt instructs the Gemma 3 4B model to act as a KMITL Budget Expert. By providing the model with "Ground Truth" context, the risk of "AI Hallucination" (making up answers) is virtually eliminated, as the model is restricted to answering only based on the provided data.

5.3 Technology Stack and Design Rationale

To balance performance, cost, and privacy, the following stack was finalized:

- Streamlit (Frontend): Chosen for rapid UI prototyping and native support for Python-based data apps.
- ChromaDB (Vector Storage): An open-source, lightweight vector database that runs entirely locally, ensuring that KMITL's internal documents never leave the local environment for indexing.
- Sentence-Transformers (Local Embeddings): By running the embedding model locally, the system avoids the costs and daily quotas associated with cloud-based embedding APIs (like Gemini Embeddings).
- Gemma 3 4B (Inference Engine): Selected for its high free-tier API quota (14,400 requests/day/user), providing a scalable and cost-effective solution for sustaining the chatbot's operations.

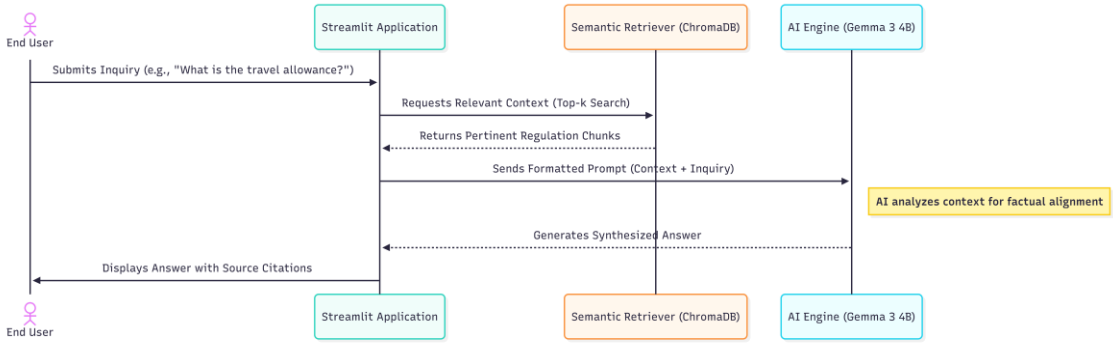

Figure Y: Sequence Diagram of the RAG-based Chatbot Interaction Flow

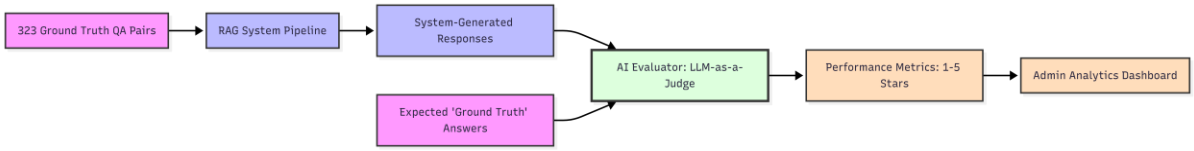

Figure Z: Automated Evaluation System Architecture using LLM-as-a-Judge Methodology.

5.4 Prompt Engineering Design

A critical component of the RAG system's accuracy is the "System Prompt." We implemented a tailored instruction set to define the AI's persona and restrict its behavior to minimize "Hallucination" (providing unauthorized or invented information).

You are the Intelligent AI Assistant of King Mongkut's Institute of Technology Ladkrabang (KMITL).
Your duty is to answer questions regarding "Budget Regulations and Procurement" with high accuracy and precision.

Instructions:
1. Answer the questions based ONLY on the information provided in the <context>. Do not fabricate information or use external knowledge.
2. If the information in the <context> is irrelevant or the answer cannot be found, reply: "I apologize, but this information is not found in the regulations."
3. Explain in a way that is easy to understand, structured as bullet points, and maintains a formal tone.

Context:
{context}

Question: {input}
Answer: {answer}

Instructional Rationale:

- Persona Identification: Defining the AI as a "KMITL Intelligent Assistant" establishes a professional identity and ensures the model remains within the relevant institutional domain.
- Contextual Grounding: Explicitly instructing the model to rely solely on the <context> tag enforces strict retrieval-based answering, preventing the model from utilizing general training data that may conflict with specific KMITL rules.
- Negative Constraint: The mandatory "Apology Message" for missing information is a key safety measure. It ensures that the system admits its limitations rather than guessing, which is critical in legal and financial contexts.
- Response Formatting: Mandating a "Bullet Point" structure improves the readability of complex regulatory clauses for the end user.

5.5 Summary of Software and Libraries Used in the Project

Category	Tool / Library	Purpose in Project
Frontend Framework	Streamlit	Building the web application and interactive dashboard.
LLM Orchestration	LangChain	Managing the RAG pipeline, chains, and document loaders.
Vector Database	ChromaDB	Storing and retrieving semantic vector embeddings locally.
PDF Extraction	PyMuPDF (fitz)	High-fidelity text extraction from regulatory PDF files.
Embedding Model	paraphrase-multilingual-MiniLM-L12-v2	Generating multilingual vectors for Thai/English semantics.
Inference Engine	Google Gemma 3 4B	Generating human-like answers based on retrieved context.
Multimodal OCR	Google Gemini 1.5 Flash	Extracting structured data from images of receipts/invoices.

Table X: Summary of Software and Libraries

Chapter 7

Retrieval Evaluation Report

Chapter 7
Challenges and Limitations

This section documents the principal technical challenges encountered during the development and deployment phases of the RAG-based chatbot system, along with the systematic resolutions applied to each. All mitigation strategies were validated through iterative testing and performance benchmarking. The section concludes with an objective evaluation of system output quality using an LLM-as-a-Judge methodology.

7.1 Technical Challenges and Resolutions

Challenge 1: LangChain Architecture Migration	
Challenge	The application crashed with a fatal ModuleNotFoundError: No module named 'langchain.chains'. This error arose because a major architectural overhaul introduced in LangChain v1.0 and later extracted legacy conversational and retrieval chain components into isolated, modular packages, rendering previously functional import paths incompatible.
Resolution	The langchain-classic and langchain-community packages were installed to bridge the dependency gap. The retrieval logic in rag_pipeline.py was subsequently refactored so that core pipeline components, specifically create_retrieval_chain and create_stuff_documents_chain, are explicitly imported from the langchain_classic.chains namespace. This ensured stable execution and maintained full compatibility with the updated framework architecture.

Challenge 2: Deprecation of the Legacy Google Generative AI SDK	
Challenge	The main application initially imported the standalone google.generativeai library for interface initialization. This triggered persistent and severe console deprecation warnings, as Google officially ended support for this legacy package in favor of newer, officially maintained SDKs.
Resolution	Upon investigation, the standalone google.generativeai package was determined to be redundant within the main dashboard (app.py). The codebase was refactored to rely exclusively on the officially supported langchain_google_genai interface. This eliminated the deprecation warnings and reduced deployment dependencies without degrading generative model performance.

Challenge 3: Threading Conflicts with Streamlit Hot-Reloading	
Challenge	During active development, the Streamlit dashboard periodically crashed with a fatal exception: RuntimeError: can't register atexit after shutdown. Root cause analysis traced this to a thread-collision between Streamlit's hot-reloading file-watching mechanism and the scikit-learn/joblib multiprocessing backend employed by the local sentence-transformers embedding model.

Resolution	This is a documented, environment-specific artifact that manifests only when the development server attempts to forcibly restart heavily threaded background processes upon code saves. The established mitigation is to manually terminate and reinitiate the Streamlit server instance via streamlit run app.py rather than depending on auto-reload when modifying underlying machine learning components. This approach completely bypasses the thread shutdown interference.

Challenge 4: Embedding API Quota Exhaustion	
Challenge	The system initially relied on the Gemini Embedding API for all vector representations. This dependency frequently reached the strict free-tier ceiling of 1,000 requests per day, interrupting the document ingestion pipeline and preventing consistent system operation.
Resolution	The system was migrated from the Gemini Embedding API to a fully local embedding approach using the sentence-transformers library. The paraphrase-multilingual-MiniLM-L12-v2 model was selected based on its demonstrated competency with Thai-language text.
Outcome	Daily API embedding restrictions were completely eliminated. The system now processes an unlimited volume of document embeddings locally and at no cost, while retaining multilingual support.

Challenge 5: Generative LLM Quota Constraints	
Challenge	The primary language model operated under a restrictive quota of 20 requests per day, rendering the system impractical for any sustained usage or iterative development workflows.
Troubleshooting	An attempt to integrate gemini-2.0-flash revealed that this model no longer offers a free tier (effective quota: 0 requests). Testing gemma-3-12b-it yielded unacceptably slow inference latency and exposed an architectural limitation: the model does not support instruction delivery via System Messages.
Resolution	The backend LLM was migrated to gemma-3-4b-it, which provides a substantially more generous free-tier allowance of 14,400 requests per day. To accommodate the model's architecture, the system prompt was refactored to be passed entirely as a Human Message, ensuring proper instruction adherence without reliance on System Message support.

Challenge 6: Inference Latency and Response Optimization	
Challenge	The chatbot's initial response generation averaged approximately 20 seconds per query. This level of latency produced a substantially degraded user experience and was deemed unacceptable for production deployment.
Resolution	The semantic search retriever was fine-tuned by reducing the k parameter, which governs the number of relevant context chunks passed to the language model, from 15 to 12. This adjustment reduced the volume of text the model must process per inference call without materially compromising retrieval coverage.
Outcome	Average response time improved from approximately 20.0 seconds to 4.28 seconds, representing a reduction of approximately 79 percent in inference latency.

Challenge 7: Streamlit Cloud Deployment Dependency Conflicts	
Challenge	Upon deploying the application to the Streamlit Community Cloud, the system encountered a fatal initialization error (TypeError originating from _message.Message._CheckCalledFromGeneratedFile()). This crash occurred due to a well-known version incompatibility between the opentelemetry package employed by chromadb and newer versions of the protobuf library pre-installed in the cloud's Python environment.
Resolution	To resolve this deployment failure, the application environment was downgraded to a more stable Python 3.11 instance via the Streamlit configuration panel. Furthermore, the protobuf library was explicitly pinned to version <=3.20.3 within the requirements.txt file. This approach ensured binary compatibility between the generated protocol buffer files and the runtime library, granting a successful and error-free cloud deployment.

Chapter 8

Future Development Roadmap

The current system represents a functional first deployment of a Thai-language Retrieval-Augmented Generation chatbot for KMITL's budget disbursement and procurement domain. It meets the performance objectives defined in Chapter 1 and is ready for institutional use. During the course of design, development, and evaluation, however, a number of limitations and opportunities for improvement were identified that fall outside the scope of the present project. This chapter documents six priority areas for future development, ordered from foundational infrastructure improvements to advanced functional extensions. Each section describes the motivation for the enhancement, the technical approach required to realise it, and the anticipated benefit to the system and its users.

8.1 Expansion of the PDF Document Corpus

The knowledge base underpinning the current system is constructed from a defined set of regulatory PDFs covering the central laws, ministerial regulations, and KMITL operational manuals directly relevant to procurement and budget disbursement. This corpus was selected to provide comprehensive coverage of the domain as defined in Chapter 1, and it reflects the documents most commonly consulted by KMITL's administrative and finance personnel in their day-to-day work.

In practice, however, individual faculties and administrative divisions within the university issue supplementary circulars, departmental procedural notes, and internal guidelines that interpret or extend the provisions of the central regulatory documents. Staff working in specialised units, such as the Graduate School, the Research and Innovation Division, or the Office of International Affairs, frequently need to cross-reference this supplementary material alongside the central regulations. The current system cannot answer questions that require knowledge of these departmental sources, which limits its utility for staff whose work is governed by faculty-level rules.

Expanding the corpus to encompass departmental documents is therefore a natural and high-priority next step. From a technical standpoint, the ingestion pipeline is already designed to accommodate this expansion. Adding new documents requires only that the relevant PDF files be placed in the data directory and the ingestion script be executed to rebuild the vector database. No changes to the application code, the embedding model, or the retrieval configuration are required.

The principal effort lies in document governance rather than technical integration. A formal process would need to be established to identify which departmental documents are authoritative and current, to obtain them from the relevant units, and to ensure that superseded or draft versions are not incorporated into the corpus. Responsibility for maintaining the corpus, including the removal of outdated documents when regulations are revised, would need to be assigned to a designated administrator. Without such a process, the corpus risks becoming inconsistent over time, which would undermine the reliability of the system's responses.

8.2 Administrative Monitoring Dashboard

In the current implementation, system performance metrics are recorded to log files and evaluated offline using the scripts described in the evaluation chapter. This approach is sufficient for periodic benchmarking and for the formal evaluation conducted as part of this project. It does not, however, provide the real-time visibility that an operational system requires. A system administrator who wishes to detect a deterioration in retrieval quality, identify categories of question that the system consistently handles poorly, or verify that a recent corpus update has improved coverage must run evaluation scripts manually and interpret their output without the benefit of visual summaries or trend analysis.

A dedicated administrative monitoring dashboard would address these limitations. The dashboard would visualise the key retrieval metrics, including Recall at k, Precision at k, and Mean Reciprocal Rank, aggregated over a configurable rolling time window. This would allow administrators to observe trends in performance over days or weeks rather than relying on single-point evaluations. A secondary view would surface individual queries for which the top retrieved chunks fell below the cosine similarity acceptance threshold, since these represent cases where the system's response may have been based on weakly relevant material and warrant manual review.

The dashboard would also expose usage analytics, including the distribution of query topics, the documents most frequently cited in responses, and the proportion of queries that triggered the insufficient context fallback response. This information would directly inform corpus curation decisions, highlighting regulatory areas that are frequently queried but currently underserved by the document set. A further panel showing the composition and coverage of the golden evaluation dataset would support the expansion effort described in the following section.

The dashboard could be implemented as an additional page within the existing Streamlit application, drawing on query logs stored in a lightweight SQLite database. This approach would preserve the system's zero-dependency philosophy while providing a materially more informative operational view than the current log-file approach.

8.3 Expansion of the Golden Evaluation Dataset

The evaluation methodology described in Chapter 6 relies on a golden dataset of representative procurement and disbursement questions, each paired with a reference answer and a citation indicating the source document and page number from which the answer is drawn. This dataset serves as the benchmark against which the system's retrieval and generation quality are measured, and its composition directly determines the validity and representativeness of the evaluation results.

The current dataset was constructed to provide broad coverage of the document corpus and to include questions of varying complexity, from simple threshold lookups to multi-step procedural queries. Its size is limited by the time available within this project, and the statistical reliability of the evaluation metrics, particularly the LLM-as-a-Judge scores, would improve with a larger and more diverse question set.

Future expansion of the dataset should be guided by an analysis of the question types that are currently underrepresented. Three categories merit particular attention. First, multi-document synthesis questions, in which a complete answer requires drawing on passages from two or more distinct source files, are not yet well represented and are among the most demanding tasks for a RAG system. Second, questions that should correctly elicit a response of insufficient context, because the answer falls outside the defined corpus or concerns a topic not covered by any of the source documents, are important for assessing the system's calibration and its resistance to generating plausible but unsupported answers. Third, questions involving numerical comparison, such as distinguishing between procurement thresholds that apply at different transaction values, require precise retrieval and are a common source of error in practice.

The most reliable way to ensure that the expanded dataset reflects questions of genuine operational relevance is to solicit contributions from experienced KMITL finance and procurement staff. These individuals encounter the practical questions that the system is intended to answer and are well positioned to identify the scenarios that most frequently cause difficulty. A structured elicitation process, supported by clear annotation guidelines specifying the required format for questions, reference answers, and citations, would be necessary to maintain consistency across contributors and to ensure that the resulting dataset is suitable for automated evaluation.

8.4 Hybrid Search Integration

The retrieval mechanism in the current system is based entirely on dense vector similarity search. A user's question is converted to a vector embedding by the locally-hosted bge-m3 model, and the k document chunks whose embeddings are most similar to the query embedding under cosine distance are returned as context. This approach is highly effective for semantically paraphrased questions, in which the user's phrasing differs from the terminology used in the source document but conveys the same meaning. The embedding space captures these semantic relationships well.

However, dense retrieval has a known weakness with queries that hinge on precise lexical matches. In the procurement and disbursement domain, this weakness is particularly relevant for queries involving specific regulation identifiers, such as the Government Procurement and Supply Management Act B.E. 2560, for queries specifying exact monetary thresholds that distinguish between procurement methods, and for Thai financial terminology that has no close semantic neighbour in the embedding space. For these queries, a chunk that contains the exact term the user is asking about may not receive the highest cosine similarity score, and the most relevant passage may therefore be ranked below less relevant but thematically adjacent passages.

Hybrid search addresses this limitation by combining the dense vector retrieval pipeline with sparse lexical retrieval using the BM25 algorithm. BM25 ranks document chunks according to the frequency and inverse document frequency of the query terms they contain, making it naturally suited to exact-match and terminology-sensitive queries. The results of the two retrieval methods are merged using reciprocal rank fusion, which produces a unified ranked list by combining the rank positions of each chunk in the two result sets without requiring their scores to be on a common scale.

Implementing hybrid search within the existing architecture would require maintaining a BM25 index over the document chunks in parallel with the ChromaDB vector index, and modifying the retriever component to query both indexes for each incoming question before applying the fusion function. The additional storage overhead is modest, since BM25 indexes are sparse and lightweight compared to dense vector stores. The computational overhead at query time is also low, as BM25 scoring is substantially faster than vector similarity search. The expected improvement in retrieval accuracy for regulation-specific and threshold-sensitive queries justifies this additional complexity.

8.5 Receipt Verification and Automated Document Submission

The current system provides informational guidance on procurement and disbursement procedures. It answers questions about what documents are required, what thresholds apply, and what steps must be followed, but it does not interact with the actual documents that staff produce in the course of carrying out those procedures. A natural and high-value extension would be to introduce a document verification capability that allows the system to inspect submitted receipts and invoices against the procedural requirements it already understands.

Under the current procurement procedures at KMITL, staff are required to attach supporting documents, including original receipts, tax invoices, and vendor certifications, to their disbursement claims before submitting them for approval. Verifying that these attachments are present, legible, correctly formatted, and internally consistent with the claimed amounts is a manual task performed by finance personnel. Errors at this stage, such as a missing document field, an inconsistency between the claimed amount and the receipt total, or the submission of a document type that does not satisfy the regulatory requirement for a given procurement category, result in claims being returned for correction, adding delays to procurement cycles.

An optical character recognition-based verification component would allow staff to photograph or scan a receipt and submit it directly through the chatbot interface. The system would apply a Thai-language OCR engine to extract key fields from the document, including vendor name, tax identification number, transaction date, item descriptions, and the total amount inclusive of value-added tax. These extracted values would then be compared against the requirements specified in the relevant procurement regulation retrieved from the knowledge base. If the document satisfies all applicable requirements, the system would confirm its validity and, where integration with KMITL's internal document management system is available, initiate an automated submission workflow routing the document to the appropriate approving officer.

Realising this extension requires three components that go beyond the current system's architecture. First, a Thai-language OCR engine capable of handling the varied fonts, layouts, and print qualities found in commercial receipts must be integrated. Second, field extraction logic tailored to common Thai receipt and invoice formats must be developed, since the layout conventions differ from those assumed by general-purpose document understanding models. Third, an authenticated API connection to KMITL's internal document management or workflow system must be established. Given this additional complexity, the extension also requires a formal review of data handling and privacy requirements, since the submitted documents may contain commercially sensitive or personally identifiable information.

8.6 Continuous Evaluation and Model Maintenance

The evaluation conducted in Chapter 6 provides a rigorous baseline assessment of system performance at the point of initial deployment. It establishes that the system meets its defined retrieval and generation quality targets against the golden dataset constructed for this project. What it cannot establish is how performance will evolve as the system is used in production over months and years, as the regulatory documents it draws on are revised, and as the informational needs of KMITL's administrative personnel change.

Three types of drift pose a risk to sustained performance. First, regulatory drift occurs when the source documents are amended and the vector database is rebuilt to incorporate the updated content. Although the core retrieval mechanism remains unchanged, the distribution of document passages shifts, and questions that were previously answered well may be affected if the relevant passages have moved, been restructured, or been replaced by provisions with different wording. Second, query drift occurs as staff become familiar with the system and begin asking more complex or boundary-testing questions that are not represented in the original golden dataset. The evaluation metrics computed against the original dataset may remain stable while the system's actual performance on live queries deteriorates. Third, model drift can occur if the embedding model or the language model API is updated by its provider, changing the characteristics of the representations or completions the system produces.

A programme of continuous evaluation would mitigate all three risks. The programme would operate by periodically sampling a random subset of live queries from the query log, obtaining reference answers for the sampled questions from domain experts within KMITL's finance and

procurement team, and incorporating the resulting question-answer pairs into the golden dataset. The automated evaluation scripts would then be re-run against the expanded dataset to produce updated metric estimates. Where the updated metrics reveal a statistically meaningful regression relative to the baseline, the responsible component can be investigated, adjusted, and re-evaluated.

Beyond regression detection, the continuous evaluation programme would also generate ongoing evidence of the system's value to the institution, providing administrators with documented performance trajectories that can inform decisions about corpus expansion, model upgrades, and resource allocation. The infrastructure required is modest: a scheduled script to sample and log queries, a lightweight interface for domain expert annotation, and version-controlled storage for the growing golden dataset. These components are compatible with the system's existing architecture and would require no changes to the deployed application.

Chapter 9

Conclusion and Future Work